# Pause: Claude Code Build Guide

Oct 25, 2026 · @Saurav

## Project overview and goals

Pause is a Chrome extension that gently interrupts gambling sessions by tying the user's gambling time to the health of a virtual pet, with all AI running locally through Gemma 4 E2B on Ollama. Paste this whole document into Claude Code as context, then feed it the step prompts one at a time.

**Who it helps:** people with a gambling addiction who want a nudge to stop and think, not a hard block.

**Core idea:** the user sets a daily gambling-time budget (for example 2 hours). Every active minute on a gambling site drains the pet's health in proportion to that budget. Time away from gambling heals the pet. At low health, a gentle pause overlay appears with the user's own reason for quitting and a helpline link.

### Design principles

- Minimally invasive: a small corner widget by default; the full overlay only at low health or on a bet or deposit click.
- Never punitive: the pet never dies permanently, and the language is warm and non-judgmental.
- Private by design: page text is classified on the user's machine and never sent anywhere. This is the headline of the demo.
- Fast: heuristics first, Gemma only for unknown domains, results cached per domain.
- Always escapable: "Continue anyway" is never hidden, and a helpline link is always one click away.

### Hackathon success criteria (7 hours total)

1. A known betting site (for example draftkings.com) shows the pet widget and the health bar drops while the user is active.
2. An unknown gambling-like site is caught by Gemma E2B and cached.
3. A normal sports news page stays quiet, with no false positive.
4. The pet changes through 4 stages and recovers when the user leaves.
5. Demo mode compresses time so all of the above shows in under 2 minutes.

## Ground rules for Claude Code

Put this section at the top of a `CLAUDE.md` file in the project root so Claude Code reads it on every step.

- Build a Chrome extension on Manifest V3 using plain JavaScript (ES modules where supported). No bundler, no React, no npm build step. The folder must load directly through chrome://extensions with "Load unpacked".
- Keep dependencies at zero. Use only browser APIs and `fetch` to Ollama.
- The model is `gemma4:e2b` served by Ollama at `http://localhost:11434`. Make the model name a single constant in `config.js`.
- Service workers in MV3 are killed after about 30 seconds idle. Never use `setInterval` or in-memory state for anything that must persist. Use `chrome.alarms` and `chrome.storage.local`.
- Never let a failed Ollama call break the page. Every Gemma call needs a timeout (8 seconds), a try/catch, and a fallback (hardcoded lines or a keyword-only decision).
- Inject all page UI inside a Shadow DOM so the host site's CSS cannot affect it.
- Tone of all user-facing text: warm, short, non-judgmental. Never shame, never use the word "addict".
- Work in small steps. After each step: list the files changed, tell me how to test it in Chrome, and stop. Do not start the next step.
- Prefer simple and working over clever. This is a 7-hour hackathon project.

**Ollama gotcha (do this before testing any Gemma step).** Ollama rejects requests from extension origins unless told otherwise. Quit any running Ollama menu-bar app, then start it from a terminal:

```
OLLAMA_ORIGINS="chrome-extension://*" ollama serve
ollama pull gemma4:e2b
```

Verify with `curl http://localhost:11434/api/tags`. If the extension still gets a 403, the origin setting did not apply.

## Phase 1: Scaffold, detection, and time tracking (about 2 hours)

Goal of this phase: a working extension with no AI at all that detects known gambling sites and drains a health value while the user is active on them. Paste each prompt below into Claude Code, one at a time.

## Step 1: Scaffold and shared config

```
Read CLAUDE.md and the project overview. Create a Manifest V3 Chrome
extension called "Pause" in this folder with these files: manifest.json,
background.js, content.js, popup.html, popup.js, popup.css, config.js,
storage.js, and an icons folder with simple placeholder PNGs (16, 48, 128).

manifest.json permissions: storage, alarms, idle, tabs, activeTab.
host_permissions: <all_urls> and http://localhost:11434/*. Register
background.js as a module service worker, content.js as a content script on
<all_urls> at document_idle, and popup.html as the action popup.

config.js exports constants: MODEL_NAME = "gemma4:e2b", OLLAMA_URL =
"http://localhost:11434/api/generate", DEFAULT_BUDGET_MINUTES = 120,
DEMO_MULTIPLIER = 10, RECOVERY_PER_HOUR = 5.

storage.js exports small async helpers wrapping chrome.storage.local:
getState(), setState(partial), and a default state shape: { health: 100,
gamblingMinutesToday: 0, budgetMinutes: 120, lastResetDate: <today ISO date>,
streak: 0, reason: "", demoMode: false, enabled: true, cache: {}, cooldowns:
{}, pausesThisWeek: 0, lastGamblingAt: null }.

Do not build any features yet. Tell me how to load the extension unpacked and
confirm it loads with no errors.
```

## Step 2: Domain list and page detection

```
Add fast, non-AI gambling detection.

1. Create domains.js exporting an array of at least 60 known gambling domains (sports betting, online casinos, poker, lotteries, daily fantasy, crypto casinos): for example draftkings.com, fanduel.com, betmgm.com, caesars.com, espnbet.com, bet365.com, stake.com, pokerstars.com, powerball.com, megamillions.com, and state lottery sites. Export a function isKnownGamblingDomain(hostname) that matches the domain or any subdomain.

2. Create a keyword scorer in content.js: scan document.title, meta description, and the first 3000 characters of document.body.innerText for terms like parlay, odds, sportsbook, place bet, bet slip, deposit bonus, free spins, jackpot, slots, roulette, blackjack, lottery tickets. Return a score 0 to 1 and the matched terms. Strong match means 3 or more distinct terms.

3. On page load, content.js sends a message { type: "PAGE_SIGNAL", hostname, url, title, keywordScore, textSample } (textSample = first 1500 characters of visible text) to background.js. background.js replies with { gambling: boolean, source: "domain" | "cache" | "unknown" }.

4. In background.js, if the hostname is a known domain, mark it gambling immediately and write it to state.cache[hostname] = { gambling: true, source: "domain", at: Date.now() }. If it is not known and keywordScore is below 0.3, cache it as not gambling. Otherwise reply "unknown" (Gemma will handle this in Phase 2).

Add console logs so I can see decisions in the service worker console. Tell me how to test on draftkings.com and espn.com and what logs to expect.
```

### Step 3: Active-time tracking, health, and badge

```
Implement the time tracker in background.js using chrome.alarms, not
setInterval.

- Create an alarm named "tick" that fires every 1 minute (set up on install
and on service worker startup if missing).
- On each tick: check chrome.idle.queryState(60). If not "active", do
nothing. Otherwise get the active tab in the focused window, read its
hostname, look up state.cache[hostname]. If gambling is true, call
addGamblingMinute(). If not, call maybeRecover().
- addGamblingMinute(): first run a daily reset check (if lastResetDate is not
today, reset gamblingMinutesToday to 0, and increment streak only if
yesterday had 0 gambling minutes, else set streak to 0, then set
lastResetDate to today). Then add 1 minute (multiply by DEMO_MULTIPLIER if
state.demoMode). Recompute health = max(0, 100 - gamblingMinutesToday /
budgetMinutes * 100). Save lastGamblingAt = now.
- maybeRecover(): if at least 10 minutes have passed since lastGamblingAt,
add RECOVERY_PER_HOUR / 60 health per tick (times DEMO_MULTIPLIER in demo
mode), capped at 100.
- After every change, update the toolbar badge with
chrome.action.setBadgeText (show the rounded health number) and
setBadgeBackgroundColor (green at 80 and up, yellow at 50 to 79, orange at 20
to 49, red below 20).

Export a pure function computeHealth(minutes, budget) and a
getPetStage(health) returning "happy" | "tired" | "sick" | "asleep" using the
thresholds 80, 50, 20. Add a quick manual test: a debug message type
"DEBUG_ADD_MINUTES" I can call from the service worker console.

Tell me how to verify the badge changes while sitting on a gambling site, and
how the idle check behaves.
```

## Phase 2: Gemma classification and pet state (about 1.5 hours)

Goal of this phase: unknown domains get classified locally by Gemma E2B and cached, and the background worker exposes a clean pet state that the UI can read.

## Step 4: Gemma classifier through Ollama

```
Create gemma.js (imported by background.js) with two exported functions.

1. classifyPage({ hostname, url, title, textSample }):
   - POST to OLLAMA_URL with { model: MODEL_NAME, stream: false, format:
"json", options: { temperature: 0, num_predict: 120 }, prompt }.
   - The prompt: "You classify webpages. Decide if this page is primarily
about or offering real-money gambling (sports betting, casino games, poker,
lotteries, daily fantasy with cash entry, crypto casinos). News, scores, and
general sports coverage are NOT gambling. Reply with only JSON:
{\"gambling\": true or false, \"type\": \"sports_betting\" | \"casino\" |
\"lottery\" | \"poker\" | \"other\" | \"none\", \"confidence\": number from 0
to 1}. Page title: ... URL: ... Text: ..." (fill in the fields, text capped
at 1500 characters).
   - Use AbortController with an 8 second timeout. Parse the JSON safely
(strip code fences if present). On any failure return { gambling: false,
type: "none", confidence: 0, failed: true } and do NOT cache failures.
   - Treat the page as gambling only when gambling is true AND confidence is
at least 0.6.

2. checkOllamaHealth(): GET http://localhost:11434/api/tags and return
whether MODEL_NAME is present.

Wire it into the PAGE_SIGNAL handler from Step 2: when the result was
"unknown", call classifyPage, cache the result by hostname (with type and
confidence), and reply to the content script. Avoid duplicate in-flight
classifications for the same hostname with a Map of pending promises. Log
timing so I can see how long E2B takes.

Tell me how to test using a page with gambling keywords that is not in the
domain list, and how to confirm the cache hit on a second visit.
```

### Step 5: Pet state API and stage-change events

```
Create pet.js (imported by background.js) as the single source of truth for
pet logic, and expose it to the UI by message passing.

- getPetSnapshot(): returns { health, stage, minutesToday, budgetMinutes,
streak, reason, enabled, demoMode, isGamblingNow } where isGamblingNow
reflects the active tab's cache entry.
- Message handlers in background.js: "GET_PET" returns the snapshot;
"SET_SETTINGS" accepts { budgetMinutes, reason, enabled, demoMode } with
validation (budget 15 to 600); "RESET_DAY" for debugging.
- Detect stage changes inside addGamblingMinute() and maybeRecover(): when
the stage differs from the previous stage, store lastStage and broadcast {
type: "PET_UPDATE", snapshot, stageChanged: true, direction: "worse" |
"better" } to the active tab through chrome.tabs.sendMessage (ignore errors
if no listener).
- Also broadcast a plain PET_UPDATE on every tick so a widget can animate the
health bar.

No UI yet. Add a debug command so I can set health to any value from the
service worker console and see the stage-change broadcast in a test tab.
Summarize the full message contract (message types and payload shapes) at the
top of pet.js as a comment, because the UI steps will rely on it.
```

## Phase 3: Pet art, widget, popup, and pause overlay (about 2 hours)

Goal of this phase: everything the judges will see. Spend design effort here.

## Step 6: Pet art as reusable SVG

```
Create pet-art.js exporting renderPet(stage, size) which returns an SVG
string of a simple, friendly round creature (a blob with big eyes, think a
small cloud or cat-like blob) in 4 expression states:
- happy: open smile, bright eyes, gentle bounce animation (CSS keyframes
inside the SVG)
- tired: half-closed eyes, small yawn, slow sway
- sick: pale green tint, droopy eyes, wavy mouth, small sweat drop, still
- asleep: closed eyes, "zzz" floating, body slightly flattened

Use only inline SVG and CSS animation, no images and no libraries. Color
changes by stage through CSS variables. Also create pet-demo.html (a local
dev page, not part of the extension) that shows all 4 states side by side so
I can review the art in a browser.
```

## Step 7: Floating widget on gambling pages

```
Implement the in-page widget in content.js (put the UI code in widget.js and
load it as an additional content script).

- Create a Shadow DOM host element fixed to the bottom-right corner (z-index
max, pointer-events only on the widget itself). Inside: the pet (renderPet),
a thin health bar, a small text line ("Today: 42 min of 120"), and a collapse
button that shrinks it to just the pet face.
- Show the widget ONLY when the background reports isGamblingNow for this
page. Ask via GET_PET on load and listen for PET_UPDATE messages. Remove the
widget if the page is no longer classified as gambling.
- Animate the health bar width smoothly on each update. On stageChanged, play
a short transition (pet swaps expression, a small speech bubble appears for 6
seconds).
- The speech bubble text for now comes from a hardcoded map of 3 lines per
stage in lines.js. (Gemma-written lines come in Step 10.)
- Remember the collapsed or expanded choice in chrome.storage.local.

Style: soft rounded card, subtle shadow, readable on both light and dark
sites, never covers more than about 200 by 120 pixels. Tell me how to test on
draftkings.com.
```

## Step 8: Popup with settings and pet view

```
Build the popup (popup.html, popup.js, popup.css), width 340px.

Top: large pet (renderPet), health bar, stage label, and a streak line ("3 gambling-free days"). Middle: "Today: X of Y minutes". Settings below: a budget slider (15 to 600 minutes, shows hours and minutes), a text area "Why do you want to cut back?" (max 140 characters, saved as reason), an on/off toggle, and a Demo mode toggle tucked into a small "Advanced" disclosure with the label "1 minute counts as 10".

On first run (reason is empty), show a short onboarding card instead: one welcoming sentence, the budget slider, and the reason box, with a "Start" button. Use GET_PET and SET_SETTINGS from the message contract. Include a small "Need to talk to someone? Call 1-800-GAMBLER" footer link. Warm color palette, clean, no clutter.
```

## Step 9: The pause overlay

```
Build overlay.js (content script) for the full-screen pause moment. Use a Shadow DOM and a centered card over a dimmed backdrop.

Trigger conditions (send from background in PET_UPDATE as showOverlay: true): the pet just entered the "asleep" stage, OR the user clicks an element on a gambling page whose text or aria-label matches /place bet|bet now|deposit|add funds|buy tickets|spin/i (use a capturing click listener on document and show the overlay before the action completes, calling preventDefault once, then letting the user proceed if they choose to continue).

Overlay content: the sleepy pet, the line "Let's take a breath.", the user's own reason shown as a quote ("You said: ..."), a 10 second breathing animation (circle expanding and shrinking) during which "Continue anyway" is disabled and shows a countdown, a primary button "Close this tab", a secondary "Continue anyway" (enabled after 10 seconds), and the helpline text "1-800-GAMBLER".

On "Continue anyway", set a 30 minute cooldown for that hostname in state.cooldowns so the overlay does not return, and increment pausesThisWeek. On "Close this tab", send a message so background.js closes the tab with chrome.tabs.remove and also increments pausesThisWeek. Make sure the overlay can never trap the user: Escape key also dismisses it with the same cooldown as Continue.
```

# Phase 4: Gemma pet voice, demo mode, and polish (about 1.5 hours)

Goal of this phase: make the Gemma integration visible to users, make the demo reliable, and harden edge cases.

## Step 10: Gemma-written pet lines

```
Add pet voice generation to gemma.js: generatePetLine({ stage, direction, pageType, reason }).

Prompt Gemma (temperature 0.7, num_predict 60, 8 second timeout, NOT json mode) with a fixed persona: "You are a small, gentle pet that lives in the user's browser. You care about the user. Speak in first person, warm, never judgmental, never preachy. Maximum 20 words. One sentence. No emojis." Include the pet's stage, whether things are getting worse or better, the type of site (sports betting, casino, lottery), and the user's own reason for cutting back if present. Example target: "I'm getting sleepy... remember you're saving for the car?"

Rules: trim and strip quotes, reject any output over 30 words or containing the words "addict" or "loser", and fall back to a random line from lines.js on any failure. Generate a line only on stage changes (not every tick), cache the last line per stage in state, and include it in the PET_UPDATE broadcast as line. Update widget.js and overlay.js to display it when present. Tell me how to test by forcing a stage change from the console.
```

## Step 11: Demo mode and a scripted demo helper

```
Finish demo mode. When demoMode is on: one tick counts as DEMO_MULTIPLIER minutes of gambling time, recovery also runs at the multiplier, and the Ollama classification still runs for real.

Add a visible "DEMO" tag on the widget and popup when it is active. Add a hidden developer panel in the popup (opened by clicking the pet 5 times) with buttons: Set health to 90 / 60 / 30 / 10, Reset today, Clear classification cache, Simulate gambling minute. This lets me recover quickly during the live demo if something goes wrong. Make sure none of these controls are visible in normal mode.
```

### Step 12: Hardening and false-positive pass

```
Review the whole codebase and fix edge cases. Specifically:

- Never run on chrome://, edge://, file://, the Chrome Web Store, or
localhost.
- Skip classification on pages with fewer than 200 characters of text, and
re-check single-page apps by listening for URL changes (history.pushState
wrapper or a 2 second mutation debounce) once per session.
- Add an allowlist of obviously safe domains (espn.com, nfl.com, nba.com,
mlb.com, bbc.com, wikipedia.org, google.com, youtube.com, reddit.com) that
never trigger, even with keyword hits.
- Handle Ollama being offline: show a small non-intrusive note in the popup
("Local AI is offline. Known sites still work.") and rely on heuristics only.
- Ensure the badge, widget, and stored state stay consistent after the
browser restarts and after the service worker is killed and revived.
- Remove console noise, keep only essential logs behind a DEBUG flag in
config.js.

Then write a short README.md with install steps, the Ollama command
(including OLLAMA_ORIGINS), and a 5-line explanation of how it works. List
any remaining known issues honestly.
```

### Step 13: Demo script and slides outline

```
Write DEMO.md with a 3-minute demo script for the judges. Structure: 20-
second problem statement, 30 seconds on the local-AI privacy angle (nothing
leaves the laptop), then a live walkthrough: (1) onboarding with a personal
reason, (2) a known betting site where the widget appears and health drops in
demo mode, (3) an unknown gambling-like page that only Gemma catches, (4) the
ESPN scores page that correctly stays quiet, (5) the pet reaching the sleepy
stage and the pause overlay with the user's own reason, (6) leaving the site
and the pet recovering. End with 30 seconds on future work (WebGPU in-browser
Gemma so no install is needed, an accountability partner, and spending-limit
integrations). Add a backup plan section for what to do if Ollama or the
network fails during the demo.
```

## Time budget and troubleshooting

Budget the 7 hours like this, and cut scope from the bottom if you fall behind:

1. 0:00 to 0:30: install Ollama, pull `gemma4:e2b`, set up the repo, put this document and `CLAUDE.md` in place.
2. 0:30 to 2:30: Phase 1 (Steps 1 to 3). Checkpoint: badge changes on a betting site.
3. 2:30 to 4:00: Phase 2 (Steps 4 and 5). Checkpoint: Gemma classifies an unknown page and the result is cached.
4. 4:00 to 6:00: Phase 3 (Steps 6 to 9). Checkpoint: widget, popup, and overlay all work.
5. 6:00 to 6:40: Phase 4 (Steps 10 to 12). If short on time, skip Step 10 and keep the hardcoded lines.
6. 6:40 to 7:00: Step 13, rehearse the demo once, record a backup video.

| Symptom | Likely cause | Fix |
|---|---|---|
| Gemma call returns 403 | Ollama blocks extension origins | Quit the menu-bar app, run `OLLAMA_ORIGINS="chrome-extension://*" ollama serve` |
| Gemma call fails with a network error | Ollama not running or model not pulled | `curl http://localhost:11434/api/tags` and `ollama pull gemma4:e2b` |
| Health stops updating after a few minutes | Timer uses `setInterval` in the service worker | Use `chrome.alarms` only (Step 3) |
| Pet drains while the laptop is untouched | Idle check missing | Confirm `chrome.idle.queryState(60)` gates every tick |
| Widget styles look broken on a site | Host CSS leaking in | Render all page UI inside a Shadow DOM |
| Classification is slow or unreliable | Page text too long or prompt too loose | Cap text at 1500 characters, keep temperature 0, keep JSON mode on |
| Normal sports pages trigger the pet | Keyword score too sensitive | Add to the allowlist and require confidence of at least 0.6 |
| Gemma output is unparseable JSON | Model added prose or code fences | Strip fences, retry once, then fall back to heuristics |

**If Claude Code goes off track:** tell it to stop, re-read `CLAUDE.md`, and redo only the current step. Keep each step in its own git commit so you can roll back cleanly.